

Trabalho de Banco de Dados 1 - Praestitia

João Luis Almeida Santos

25 de junho de 2026

Índice

1	Introdução	2
2	Praestitia	2
3	Modelo Conceitual	2
4	Modelo Lógico	3
5	Modelo Físico	5
5.1	Eficiência	7
6	Consultas SQL	7
6.1	Cadastro de novo lead	7
6.2	Listagem de leads com o usuário responsável	7
6.3	Contagem de leads por origem	8
6.4	Usuários com mais de 2 leads cadastrados	8
6.5	Leads cujo email já consta como cliente	8
6.6	Distribuição percentual de clientes por status	9
6.7	Concorrentes com endereço cadastrado e seu responsável	9
6.8	Atualização do status de um lead	9
6.9	Remoção de leads perdidos	9
6.10	Funil de vendas agrupado por etapa	10
6.11	Leads com pontuação acima da média geral	10
6.12	Todos os leads, com endereço quando disponível	10
6.13	Usuários e seus grupos	10
6.14	Clientes por usuário responsável	11
6.15	Empresas com mais de um lead cadastrado	11
7	Conclusão	11

1 Introdução

Este trabalho apresenta o projeto de banco de dados relacional que dá suporte a Praestitia, um sistema de CRM voltado ao gerenciamento de leads, clientes e concorrentes. O objetivo é exercitar a passagem do modelo conceitual para o modelo lógico, sua implementação no modelo físico em PostgreSQL e, por fim, a escrita de consultas SQL que respondam a perguntas relevantes do negócio.

O documento está organizado em quatro partes. Primeiro, descreve-se Praestitia e o domínio que atende. Em seguida, apresenta-se o modelo lógico com as relações e a justificativa de cada chave estrangeira. A terceira parte exhibe e explica cada uma das consultas SQL desenvolvidas sobre o esquema. Por fim, a conclusão recapitula os pontos principais do trabalho.

2 Praestitia

Praestitia é um CRM que organiza o funil comercial de uma empresa, acompanhando um contato desde o primeiro interesse até a conversão em cliente efetivo e também registrando os concorrentes que atuam no mesmo mercado. O sistema é multiusuário, e cada vendedor cadastrado é responsável pelos registros que cria.

O domínio é composto pelas seguintes entidades.

- **Usuário**, o vendedor que opera o sistema. É responsável pelos leads, clientes e concorrentes que cadastra e pertence a um grupo de permissões.
- **Grupo e Permissão**, que controlam a autorização. Um grupo reúne um conjunto de permissões e um usuário herda as permissões do seu grupo. A relação entre grupo e permissão é de muitos para muitos.
- **Lead**, contato que demonstrou interesse inicial, mas ainda não foi convertido. Guarda a origem do contato, o status no funil de vendas e uma pontuação que indica o quão promissor é o lead.
- **Cliente**, lead convertido ou contato que já fechou negócio. Possui um status comercial, que pode ser ativo, inativo, pendente ou suspenso.
- **Concorrente**, empresa que disputa o mesmo mercado, registrada para acompanhamento competitivo.
- **Endereço**, dados de localização reutilizáveis, referenciados por leads, clientes e concorrentes.

Esse domínio justifica as consultas apresentadas mais adiante. Medir a origem dos leads, acompanhar o funil de vendas, identificar a carga de trabalho por usuário e cruzar leads com clientes já existentes são operações típicas do dia a dia de um CRM.

3 Modelo Conceitual

O modelo conceitual representa o domínio de Praestitia em um diagrama Entidade-Relacionamento, antes de qualquer preocupação com a tecnologia do banco. Ele

mostra as entidades Permissao, Grupo, Usuario, Endereco, Lead, Cliente e Concorrente, seus atributos e os relacionamentos entre elas.

Os principais relacionamentos são os seguintes. Um Grupo reúne muitas Permissões e uma Permissão pode estar em muitos Grupos, formando um relacionamento de muitos para muitos. Cada Usuario pertence a um Grupo, e um Grupo pode ter muitos Usuarios. Um Usuario é responsável por muitos Leads, Clientes e Concorrentes, e cada um desses registros pertence a um único Usuario. Por fim, Lead, Cliente e Concorrente podem apontar para um Endereco, que pode ser compartilhado por mais de um registro.

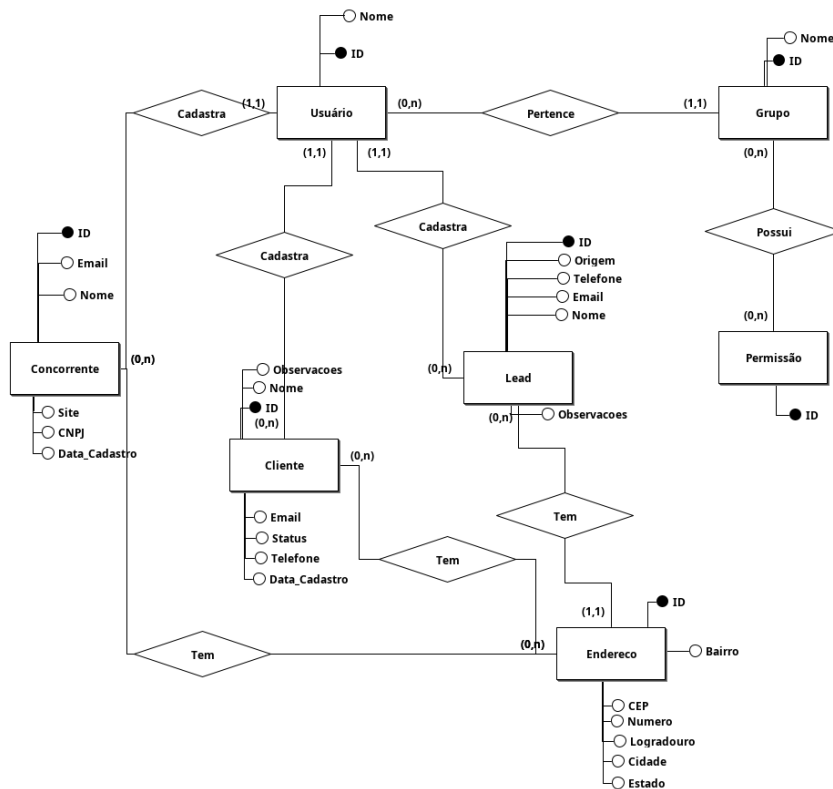


Figura 1: Modelo conceitual de Praestitia (diagrama Entidade-Relacionamento).

4 Modelo Lógico

O modelo lógico mapeia o diagrama conceitual para o modelo relacional, transformando cada entidade em uma tabela e cada relacionamento em chaves estrangeiras. Chave primária sublinhada. Chave estrangeira marcada com asterisco.

Permissao(id)

Grupo(id, nome)

Grupo_Permissao(*grupo_id, *permissao_id)

Usuario(id, nome, *grupo_id)

Endereco(id, cep, numero, logradouro, bairro, cidade, estado)

Lead(id, *usuario_id, *endereco_id, nome, email, telefone, empresa, cargo, origem, interesse, status, pontuacao, observacoes, data_cadastro, data_ultimo_contato)

Cliente(id, *usuario_id, *endereco_id, nome, email, telefone, empresa, cnpj_cpf, status, data_cadastro, observacoes)

Concorrente(id, *usuario_id, *endereco_id, nome, email, site, cnpj, data_cadastro)

Chaves estrangeiras

- **Grupo_Permissao.grupo_id referencia Grupo(id).** Um grupo pode ter várias permissões e uma permissão pode estar em vários grupos ao mesmo tempo, o que torna a relação de muitos para muitos. Como nenhum dos dois lados consegue guardar essa informação sozinho, a solução é uma tabela intermediária que registra cada combinação.
- **Grupo_Permissao.permissao_id referencia Permissao(id).** Pelo mesmo motivo, a permissão também precisa estar representada na tabela intermediária.
- **Usuario.grupo_id referencia Grupo(id).** Um usuário pertence a um grupo, mas um grupo pode ter vários usuários. A chave fica em Usuario porque é o lado que se repete na relação.
- **Lead.usuario_id referencia Usuario(id).** Um usuário pode cadastrar vários leads, mas cada lead pertence a um único usuário. A chave fica em Lead porque é o lado que se repete.
- **Lead.endereco_id referencia Endereco(id).** Pessoas do mesmo domicílio, como cônjuges, podem ser cadastradas como leads distintos mas compartilham o mesmo endereço. A chave fica em Lead para que o endereço não precise ser duplicado.
- **Cliente.usuario_id referencia Usuario(id).** Um usuário gerencia vários clientes, mas cada cliente foi cadastrado por um único usuário. A chave fica em Cliente pelo mesmo motivo do Lead.
- **Cliente.endereco_id referencia Endereco(id).** Clientes no mesmo endereço reaproveitam o mesmo registro. A chave fica em Cliente pelo mesmo motivo do Lead.
- **Concorrente.usuario_id referencia Usuario(id).** Um usuário pode monitorar vários concorrentes. A chave fica em Concorrente pelo mesmo motivo dos casos anteriores.

- **Concorrente.endereco_id** referencia **Endereco(id)**. Empresas concorrentes podem estar no mesmo endereço, como num edifício comercial. A chave fica em Concorrente pelo mesmo motivo do Lead e do Cliente.

5 Modelo Físico

O modelo físico traduz o modelo lógico no script DDL que cria as tabelas no PostgreSQL. Aqui são definidos os tipos de cada coluna, as chaves primárias e estrangeiras, e as restrições de integridade que garantem a consistência dos dados.

```
CREATE TABLE permissao (
    id SERIAL PRIMARY KEY
);

CREATE TABLE grupo (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE grupo_permissao (
    grupo_id INTEGER NOT NULL REFERENCES grupo(id) ON DELETE CASCADE,
    permissao_id INTEGER NOT NULL REFERENCES permissao(id) ON DELETE
    → CASCADE,
    PRIMARY KEY (grupo_id, permissao_id)
);

CREATE TABLE usuario (
    id BIGSERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    grupo_id INTEGER REFERENCES grupo(id) ON DELETE SET NULL
);

CREATE TABLE endereco (
    id SERIAL PRIMARY KEY,
    cep VARCHAR(10),
    numero VARCHAR(20),
    logradouro VARCHAR(200),
    bairro VARCHAR(100),
    cidade VARCHAR(100),
    estado CHAR(2)
);
```

As tabelas de leads, clientes e concorrentes guardam os contatos do funil. Note as restrições CHECK, que limitam status e origem a um conjunto fixo de valores, e o UNIQUE no e-mail do cliente, que impede cadastros duplicados.

```
CREATE TABLE lead (
    id SERIAL PRIMARY KEY,
    usuario_id BIGINT REFERENCES usuario(id) ON DELETE
    → CASCADE,
    endereco_id INTEGER REFERENCES endereco(id) ON DELETE
    → SET NULL,
    nome VARCHAR(200) NOT NULL,
```

```

email          VARCHAR(254) NOT NULL,
telefone       VARCHAR(20),
empresa       VARCHAR(200),
cargo         VARCHAR(100),
origem        VARCHAR(20) NOT NULL DEFAULT 'site',
interesse      VARCHAR(200),
status        VARCHAR(20) NOT NULL DEFAULT 'novo',
pontuacao     INTEGER NOT NULL DEFAULT 0,
observacoes   TEXT,
data_cadastro TIMESTAMPTZ NOT NULL DEFAULT NOW(),
data_ultimo_contato TIMESTAMPTZ,

CONSTRAINT lead_status_check CHECK (
    status IN ('novo', 'contatado', 'qualificado', 'em_negociacao',
    ↪ 'convertido', 'perdido')
),
CONSTRAINT lead_origem_check CHECK (
    origem IN ('site', 'indicacao', 'redes_sociais', 'email',
    ↪ 'telefone', 'evento', 'outro')
)
);

CREATE TABLE cliente (
    id          SERIAL          PRIMARY KEY,
    usuario_id  BIGINT          REFERENCES usuario(id) ON DELETE CASCADE,
    endereco_id INTEGER        REFERENCES endereco(id) ON DELETE SET
    ↪ NULL,
    nome        VARCHAR(200) NOT NULL,
    email       VARCHAR(254) NOT NULL UNIQUE,
    telefone    VARCHAR(20),
    empresa     VARCHAR(200),
    cnpj_cpf    VARCHAR(18),
    status      VARCHAR(20) NOT NULL DEFAULT 'ativo',
    data_cadastro TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    observacoes TEXT,

    CONSTRAINT cliente_status_check CHECK (
        status IN ('ativo', 'inativo', 'pendente', 'suspensao')
    )
);

CREATE TABLE concorrente (
    id          SERIAL          PRIMARY KEY,
    usuario_id  BIGINT          NOT NULL REFERENCES usuario(id) ON
    ↪ DELETE CASCADE,
    endereco_id INTEGER        REFERENCES endereco(id) ON DELETE SET
    ↪ NULL,
    nome        VARCHAR(200) NOT NULL,
    email       VARCHAR(254),
    site        VARCHAR(200),
    cnpj        VARCHAR(18),
    data_cadastro TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

As ações `ON DELETE` definem o que acontece com os registros dependentes quando o registro referenciado é removido. O `ON DELETE CASCADE` em `usuario_id` apaga os leads, clientes e concorrentes do usuário removido, enquanto o `ON DELETE SET NULL` em `endereco_id` apenas desfaz o vínculo, preservando o registro sem o endereço.

5.1 Eficiência

Para acelerar as consultas mais frequentes, o modelo físico cria índices sobre as colunas usadas em filtros e junções. Sem índice, o banco precisa varrer a tabela inteira; com índice, ele localiza as linhas diretamente.

```
CREATE INDEX idx_usuario_grupo      ON usuario(grupo_id);
CREATE INDEX idx_lead_usuario      ON lead(usuario_id);
CREATE INDEX idx_lead_email        ON lead(email);
CREATE INDEX idx_cliente_usuario  ON cliente(usuario_id);
CREATE INDEX idx_cliente_email     ON cliente(email);
CREATE INDEX idx_cliente_status    ON cliente(status);
CREATE INDEX idx_concorrente_usuario ON concorrente(usuario_id);
```

Os índices em `usuario_id` ajudam as junções entre usuário e seus leads, clientes e concorrentes. Os índices em `email` aceleram a busca por contato e a comparação entre leads e clientes. O índice em `cliente(status)` beneficia a consulta de distribuição percentual por status. As chaves primárias já recebem um índice automático do PostgreSQL, por isso não aparecem na lista.

6 Consultas SQL

Esta seção apresenta as consultas desenvolvidas sobre o esquema físico. Cada consulta é acompanhada de uma explicação do que faz e de quais recursos da linguagem SQL emprega.

6.1 Cadastro de novo lead

```
INSERT INTO lead (usuario_id, nome, email, telefone, origem, observacoes)
VALUES (1, 'Shadow Shock', 'shadowshock@elysee.fr', '+33144181818',
↵ 'indicacao', 'Presidente da Franca');
```

Insere um novo lead associado ao usuário de `id` 1. É a operação de criação do CRUD, informando apenas as colunas preenchidas e deixando as demais assumirem seus valores padrão definidos no modelo físico, como `status = 'novo'` e `pontuacao = 0`.

6.2 Listagem de leads com o usuário responsável

```
SELECT
  l.id,
  l.nome      AS lead_nome,
  l.email,
  l.origem,
  u.nome      AS responsavel
FROM lead l
```

```
JOIN usuario u ON l.usuario_id = u.id
ORDER BY l.id DESC;
```

Lista todos os leads junto com o nome do usuário responsável. Usa um JOIN entre lead e usuario pela chave estrangeira usuario_id e renomeia colunas com AS para deixar o resultado legível. O ORDER BY l.id DESC mostra primeiro os leads mais recentes.

6.3 Contagem de leads por origem

```
SELECT
    origem,
    COUNT(*) AS total_leads
FROM lead
GROUP BY origem
ORDER BY total_leads DESC;
```

Agrupa os leads por canal de origem e conta quantos vieram de cada um. Combina GROUP BY com a função de agregação COUNT(*). Responde a uma pergunta de negócio direta, indicando quais canais de captação trazem mais leads.

6.4 Usuários com mais de 2 leads cadastrados

```
SELECT
    u.id,
    u.nome AS usuario,
    COUNT(l.id) AS total_leads
FROM usuario u
JOIN lead l ON l.usuario_id = u.id
GROUP BY u.id, u.nome
HAVING COUNT(l.id) > 2
ORDER BY total_leads DESC;
```

Identifica os usuários mais ativos. Diferente do WHERE, que filtra linhas individuais, o HAVING filtra grupos já agregados. Aqui, mantém apenas os usuários com mais de dois leads, mostrando a diferença entre filtrar antes e depois da agregação.

6.5 Leads cujo email já consta como cliente

```
SELECT
    l.nome,
    l.email,
    l.origem
FROM lead l
WHERE l.email IN (SELECT email FROM cliente)
ORDER BY l.nome;
```

Encontra leads que correspondem a clientes já existentes, comparando os e-mails. Usa uma subconsulta com IN, em que a consulta interna devolve todos os e-mails da tabela cliente e a externa mantém apenas os leads cujo e-mail aparece nessa lista. Útil para detectar contatos duplicados entre as duas etapas do funil.

6.6 Distribuição percentual de clientes por status

```
SELECT
    status,
    COUNT(*)
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER (), 2)
FROM cliente
GROUP BY status
ORDER BY total DESC;
```

Mostra quantos clientes existem em cada status e qual percentual representam do total. Emprega uma função de janela. O `SUM(COUNT(*) OVER ())` soma as contagens de todos os grupos sem juntar as linhas, permitindo dividir a contagem de cada status pelo total geral. O `ROUND` limita o percentual a duas casas decimais.

6.7 Concorrentes com endereço cadastrado e seu responsável

```
SELECT
    c.nome
    c.site,
    e.logradouro,
    e.cidade,
    e.estado,
    u.nome
FROM concorrente c
JOIN endereco e ON e.id = c.endereco_id
JOIN usuario u ON u.id = c.usuario_id
ORDER BY c.nome;
```

Reúne, em uma única visão, o concorrente, seu endereço e o usuário que o cadastrou. Faz dois `JOIN` encadeados, um com `endereco` e outro com `usuario`. Como usa `JOIN` interno com endereço, só aparecem concorrentes que possuem endereço associado.

6.8 Atualização do status de um lead

```
UPDATE lead
SET status = 'contatado'
WHERE nome = 'Data' AND status = 'novo';
```

Avança o lead no funil, mudando seu status de `novo` para `contatado`. É a operação de atualização do CRUD. A condição dupla no `WHERE` garante que apenas o registro correto, e ainda no estado esperado, seja alterado.

6.9 Remoção de leads perdidos

```
DELETE FROM lead
WHERE status = 'perdido';
```

Remove os leads marcados como perdidos. É a operação de exclusão do CRUD, usada para limpar os contatos que não avançaram no funil. O `WHERE` restringe a remoção apenas aos registros com esse status.

6.10 Funil de vendas agrupado por etapa

```
SELECT
    status,
    COUNT(*)           AS total,
    ROUND(AVG(pontuacao)) AS puntuacao_media
FROM lead
GROUP BY status
ORDER BY total DESC;
```

Resume o funil de vendas. Para cada status mostra quantos leads existem e qual a pontuação média deles. Combina COUNT(*) e AVG(pontuacao) num mesmo agrupamento, dando uma visão tanto de volume quanto de qualidade por etapa.

6.11 Leads com pontuação acima da média geral

```
SELECT
    l.nome,
    l.empresa,
    l.status,
    l.pontuacao,
    u.nome AS responsavel
FROM lead l
JOIN usuario u ON u.id = l.usuario_id
WHERE l.pontuacao > (SELECT AVG(pontuacao) FROM lead)
ORDER BY l.pontuacao DESC;
```

Destaca os leads mais promissores, aqueles cuja pontuação supera a média de todos os leads. A parte (SELECT AVG(pontuacao) FROM lead) é uma consulta interna que devolve um único valor, a média, usado na comparação do WHERE. O JOIN acrescenta o responsável de cada lead.

6.12 Todos os leads, com endereço quando disponível

```
SELECT
    l.nome,
    l.status,
    e.logradouro,
    e.cidade
FROM lead l
LEFT JOIN endereco e ON e.id = l.endereco_id
ORDER BY l.nome;
```

Lista todos os leads e, quando houver, o endereço associado. Usa LEFT JOIN. Ao contrário do JOIN interno, ele mantém os leads mesmo sem endereço cadastrado, preenchendo as colunas de endereço com NULL nesses casos. Garante que nenhum lead fique de fora do relatório.

6.13 Usuários e seus grupos

```
SELECT
    u.nome AS usuario,
    g.nome AS grupo
FROM usuario u
JOIN grupo g ON g.id = u.grupo_id
```

```
ORDER BY g.nome, u.nome;
```

Mostra a qual grupo de permissões cada usuário pertence. É um JOIN simples pela chave estrangeira `grupo_id`, ordenado por grupo e depois por nome, facilitando a visualização de quem está em cada grupo.

6.14 Clientes por usuário responsável

```
SELECT
    u.nome          AS responsavel,
    COUNT(c.id)     AS total_clientes
FROM usuario u
JOIN cliente c ON c.usuario_id = u.id
GROUP BY u.id, u.nome
ORDER BY total_clientes DESC;
```

Mede a carteira de cada usuário, contando quantos clientes cada um gerencia. Agrupa os clientes por usuário com `GROUP BY` e ordena do maior para o menor, indicando quem concentra mais clientes.

6.15 Empresas com mais de um lead cadastrado

```
SELECT
    empresa,
    COUNT(*)          AS total_leads,
    MAX(pontuacao)    AS maior_pontuacao
FROM lead
WHERE empresa IS NOT NULL
GROUP BY empresa
HAVING COUNT(*) > 1
ORDER BY total_leads DESC;
```

Encontra empresas que geraram mais de um lead, indicando contas de maior interesse comercial. O `WHERE empresa IS NOT NULL` descarta leads sem empresa antes de agrupar, o `HAVING COUNT(*) > 1` mantém só os grupos com mais de um lead, e o `MAX(pontuacao)` mostra a melhor pontuação dentro de cada empresa.

7 Conclusão

O trabalho percorreu o ciclo completo de modelagem de um banco de dados relacional para o CRM Praestitia. A partir da compreensão do domínio, com usuários, grupos, permissões, leads, clientes e concorrentes, definiu-se o modelo lógico com suas chaves primárias e estrangeiras justificadas, implementou-se o modelo físico em PostgreSQL e escreveram-se consultas que respondem a perguntas reais do negócio.

As consultas exercitaram os principais recursos da linguagem SQL. Foram usadas as quatro operações do CRUD, que são `INSERT`, `SELECT`, `UPDATE` e `DELETE`, as junções com `JOIN` e `LEFT JOIN`, as agregações com `GROUP BY` e `HAVING`, as subconsultas e as funções de janela. Mais do que demonstrar a sintaxe, cada consulta foi pensada para extrair informação útil ao gerenciamento do funil de

vendas, mostrando como um esquema bem modelado sustenta as decisões do dia a dia de um sistema de CRM.